# Alexa+ Mission Execution Platform

**Comprehensive Master Project Dossier & Current State Blueprint**
*Prepared for Shanmukheswar | Hackathon 2026 Submission*

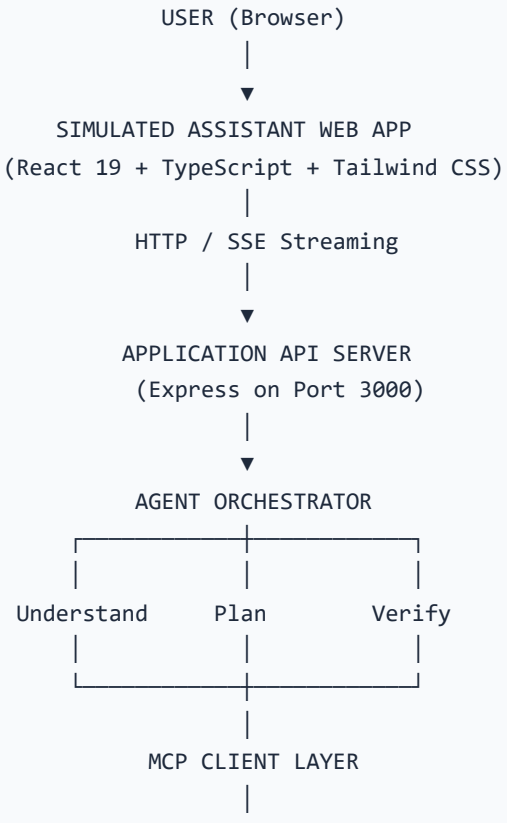
Primary Track: Alexa+ | Mini-Challenge: AWS Builder | Mini-Challenge: Open Source | 16 Test Suites: 100% Pass | TypeScript: 0 Errors

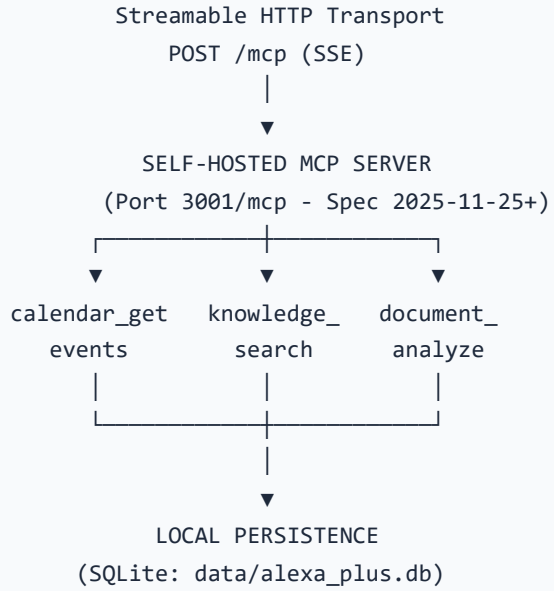
> **Purpose of this Document:** This document contains pin-to-pin technical details, architecture records, test proofs, prototype screenshots, and an inventory of completed and remaining work. When starting a fresh session, provide this document to orient the assistant in 30 seconds.

## 1. Executive Summary & Positioning

This project is an **Alexa+-inspired autonomous agentic assistant platform** built around the official **Model Context Protocol (MCP) Streamable HTTP** specification (spec `2025-11-25+`). Rather than simulating tool execution through static chat prompts, the platform connects an agent orchestrator to an independent self-hosted MCP server (`:3001/mcp`) over HTTP with dynamic capability discovery and persistent SQLite state tracking.

## 2. System Architecture Blueprint






## 3. Pin-to-Pin Feature & Reality Catalog

| Component | User Interaction | Underlying Execution | Grounding |
|---|---|---|---|
| **Natural Composer** | Prompt typing, keyboard submit (Ctrl+Enter), popular use case pills. | Generates conversation ID and mission entity via `POST /api/missions`. | SQLite relational persistence |
| **Execution Modes** | ChatGPT-style dropdown: *Quick, Deep Research, Autonomous, Build*. | Instructs orchestrator to generate 2 to 4-step execution DAGs tailored to depth. | Real DAG planning |
| **Document Attachment** | `+` and paperclip buttons upload PDF, DOCX, TXT, MD, JSON. | Calls `POST /api/knowledge/upload` and indexes into knowledge store. | SQLite knowledge table |
| **Voice Dictation** | Microphone button with animated pulse indicator. | Web Speech API (SpeechRecognition) appends text to prompt. | Browser Web Speech API |
| **Mission Workspace** | Live stages: *Understand → Plan → Tools → Verify → Result*. | Subscribes to Server-Sent Events (SSE) from `/api/missions/:id/stream`. | Real-time SSE event bus |
| **Interactive Tool Runner** | Tools page "Use" button opens runner modal. | Executes JSON arguments against MCP server via `POST /api/tools/:name/execute`. | Raw MCP JSON-RPC 2.0 |
| **Options Menu (3 Dots)** | Missions page header dropdown. | Copy ID, copy goal text, re-run mission, delete mission ( `DELETE /api/missions/:id` ). | SQLite database mutation |
| **Honest Integrations** | Integrations page configure/connect modal. | Configures API keys and email ( `shanmukheswaramedicharla@gmail.com` ). | Explicit status in SQLite |

## 4. Mini-Challenges Technical Breakdown

### A. AWS Builder Track (Amazon Bedrock Converse Integration)

- **SDK Installed:** `@aws-sdk/client-bedrock-runtime` (v3.1154.0).
- **Implementation Module:** `src/agent/llm/bedrock.mjs`.
- **Models Supported:** `anthropic.claude-3-5-sonnet-20241022-v2:0` & `amazon.nova-pro-v1:0`.
- **API Employed:** AWS Bedrock `ConverseCommand` with structured `toolConfig.tools` and message block hierarchy.
- **Zero-Crash Fallback:** If AWS credentials are absent or offline, it safely defaults to the deterministic `AutonomousDecisionEngine`.
- **Test Command:** `npm run test:bedrock` (5 tests, 100% pass).

### B. Open Source Track (`mcp-session-reaper`)

- **Standalone Package Path:** `packages/mcp-session-reaper/`.
- **License:** MIT License (standalone LICENSE file included).
- **Problem Solved:** Socket-aware scavenging for Streamable HTTP SSE connections in Node.js/Express, eliminating memory leaks when clients disconnect.
- **Package Files:** `package.json`, `index.js`, `LICENSE`, `README.md`, `test.js`.
- **Test Command:** `npm run test:reaper` (5 tests, 100% pass).

## 5. Prototype Screenshots & Visual Verification

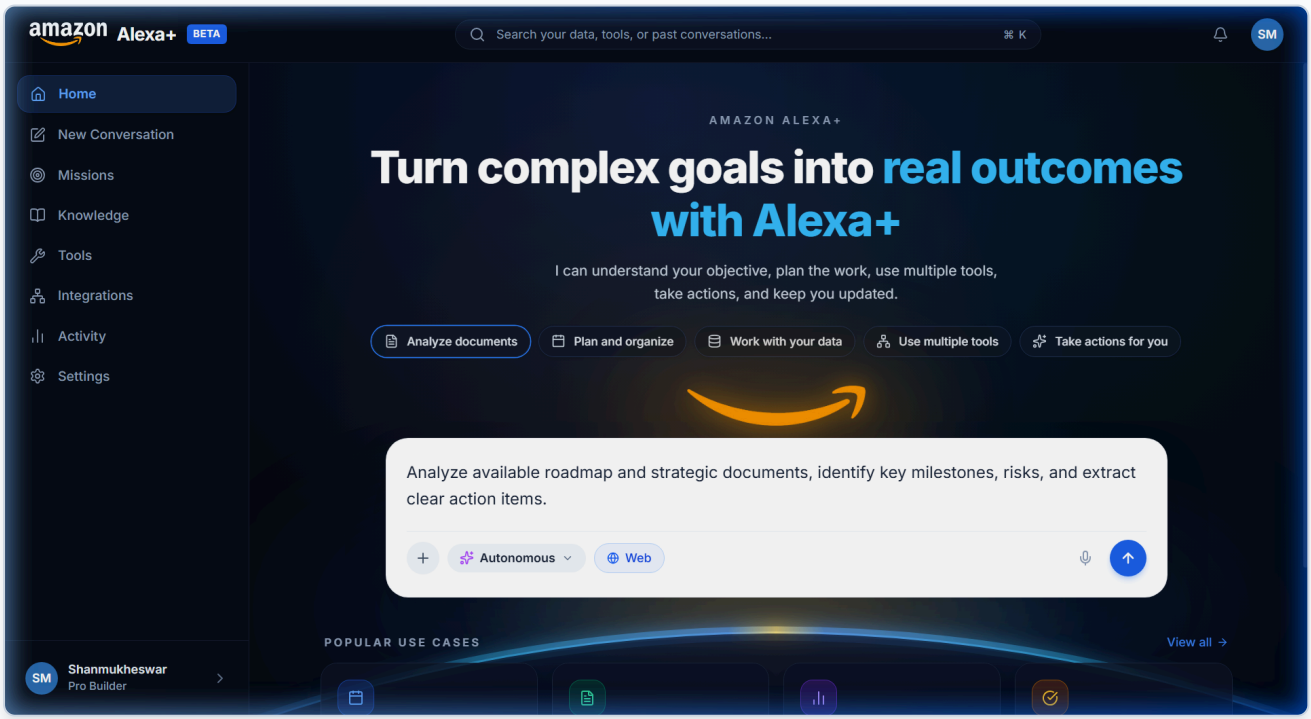



Figure 1: Production Home Page with Single-Row Capability Pills, Amazon Smile, and Prompt Composer

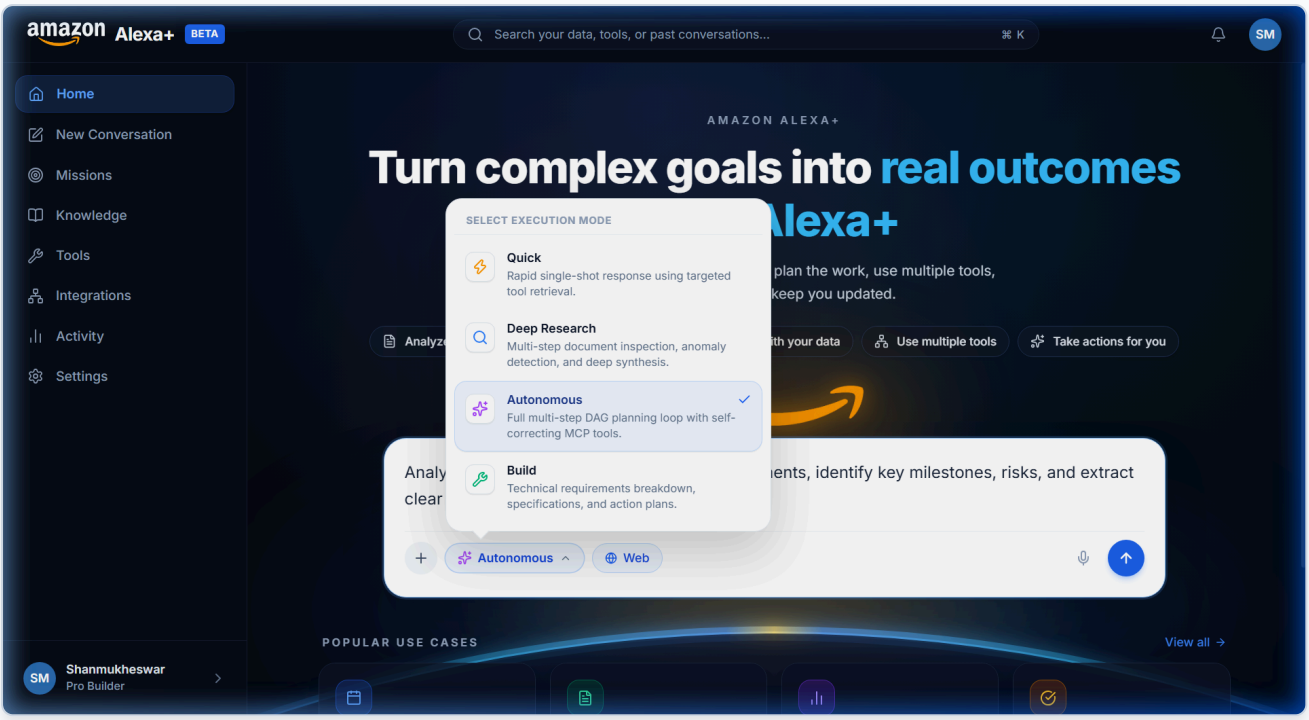



Figure 2: Upwards-Opening ChatGPT-Style Execution Modes Popover (Quick, Deep Research, Autonomous, Build)

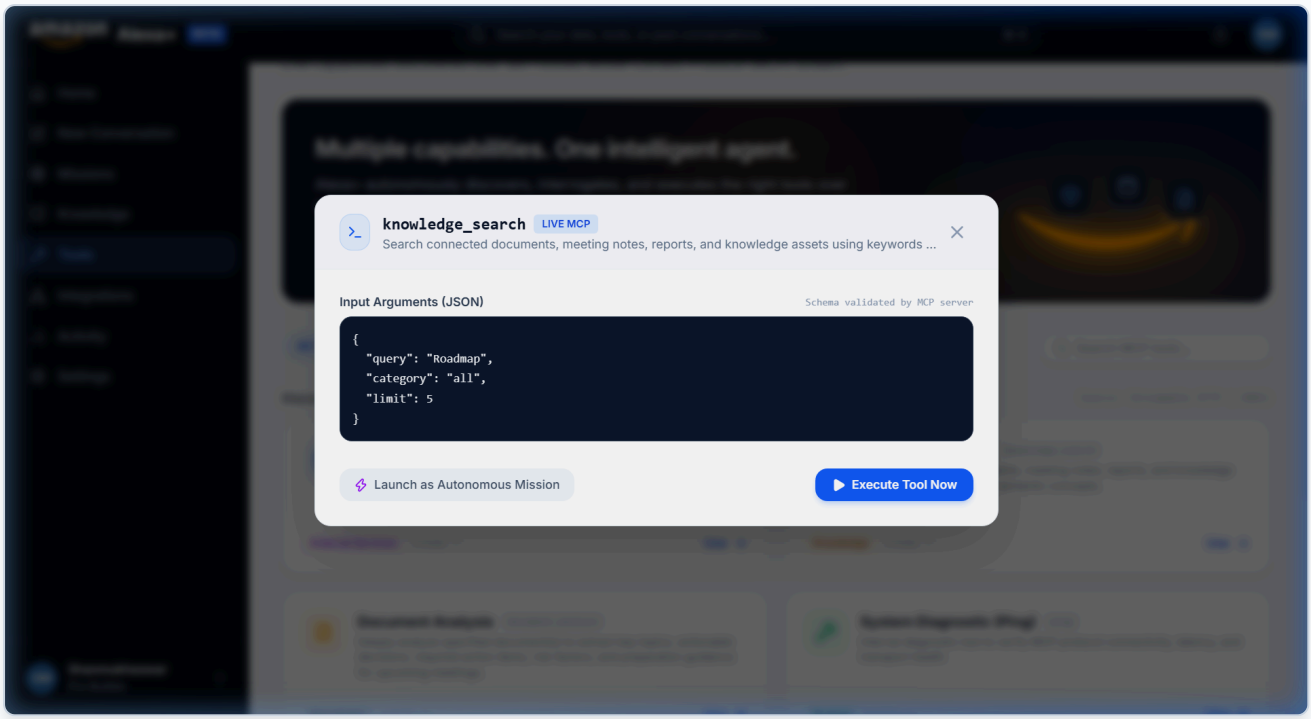



Figure 3: Tools Page Interactive MCP Tool Runner Modal with Editable JSON Arguments

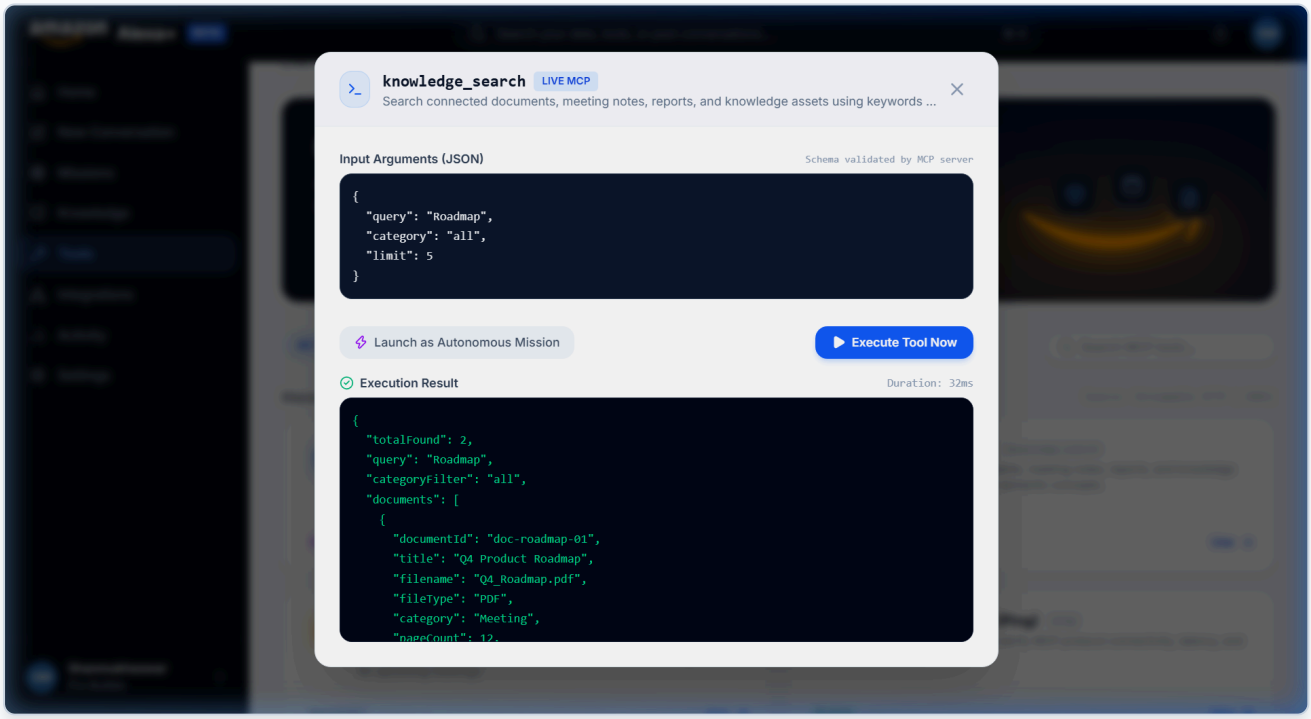



Figure 4: Live MCP Execution Terminal Output over :3001/mcp with Latency and JSON Payload

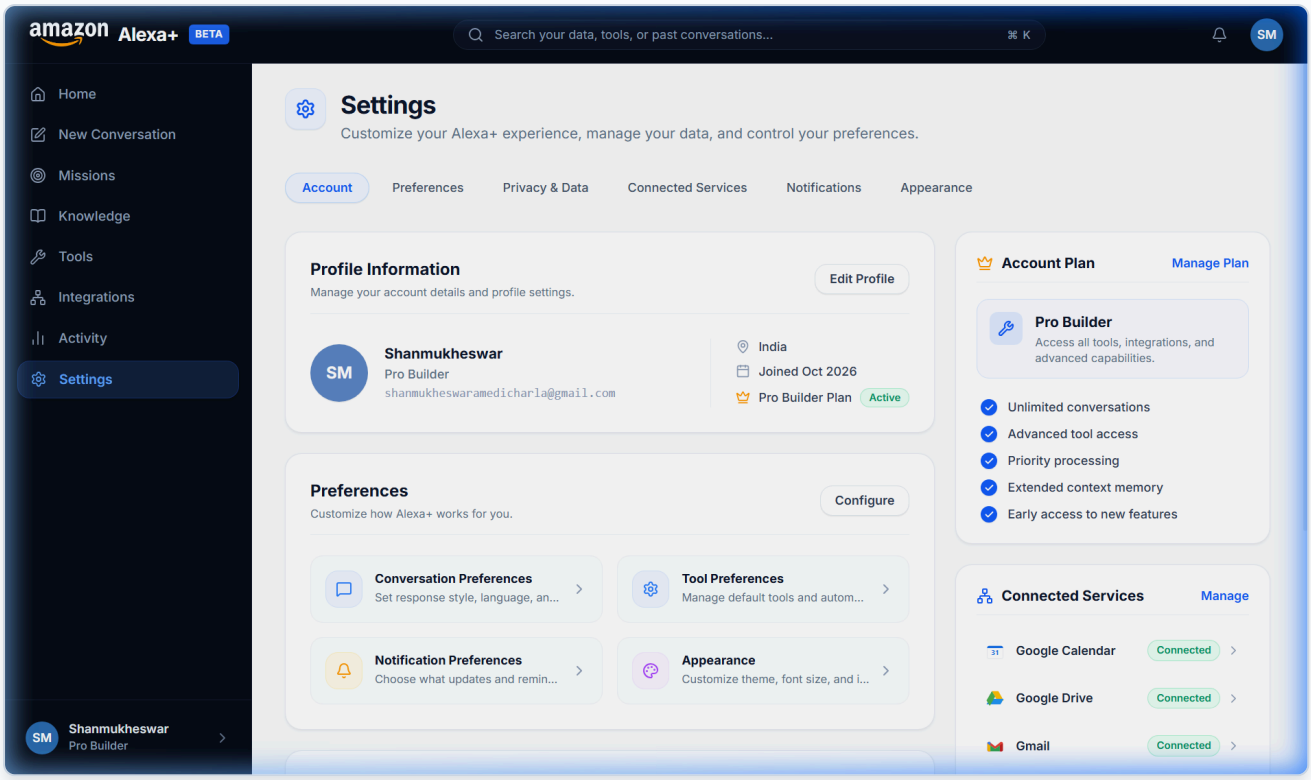



Figure 5: Settings Page displaying Runtime Configuration, Profile Identity, and Hackathon Mini-Challenges Live Verification

## 6. Automated Test Suites Verification Summary

```
Total Test Suites: 16 | Passed: 16 (100%) | Failed: 0
-----------------------------------------------------------------------
1.  tests/agent-orchestrator.test.mjs          [PASS] (MCP Lifecycle)
2.  tests/orchestrator-provider-driven.test.mjs[PASS] (Provider Decoupling)
3.  tests/openai-provider-wire.test.mjs        [PASS] (Neural Provider Wire)
4.  tests/multi-tool-chain.test.mjs            [PASS] (Multi-Tool Sequencing)
5.  tests/calendar-mcp.test.mjs                [PASS] (Calendar MCP Tool)
6.  tests/persistence-lifecycle.test.mjs       [PASS] (Durability & Recovery)
7.  tests/application-api-e2e.test.mjs         [PASS] (API & SSE Reconnection)
8.  tests/mission-workspace-real.test.mjs      [PASS] (Workspace State Integrity)
9.  tests/mcp-tools-parity.test.mjs            [PASS] (tools/list & call Parity)
10. tests/activity-persistence.test.mjs        [PASS] (Activity Timeline Store)
11. tests/knowledge-mcp-parity.test.mjs        [PASS] (Knowledge Search Tool)
12. tests/integrations-reality.test.mjs        [PASS] (Honest Capability Model)
13. tests/settings-reality.test.mjs            [PASS] (Settings Telemetry)
14. tests/hostile-stage7-audit.test.mjs        [PASS] (Hostile Attacks & Concurrency)
15. tests/aws-bedrock-integration.test.mjs     [PASS] (AWS Bedrock Converse API)
16. packages/mcp-session-reaper/test.js        [PASS] (Open Source Package Tests)
```

## 7. Inventory: What is Complete vs. What is Left Over

### ✅ What is 100% Completed & Verified

- Self-hosted MCP Server operating over Streamable HTTP on `:3001/mcp`.
- 4 executable MCP tools (`calendar_get_events`, `knowledge_search`, `document_analyze`, `ping`).
- Autonomous agent DAG orchestrator with real-time SSE streaming.
- SQLite persistence with relational tables and zero orphaned records.
- All UI pages: Home, New Conversation, Missions Workspace, Tools, Knowledge, Integrations, Activity, Settings.
- ChatGPT-style execution modes, document attachment chip badging, and speech recognition.
- AWS Bedrock Converse integration with Claude 3.5 Sonnet and Nova Pro.
- Open source `mcp-session-reaper` companion library.
- Complete Devpost submission package, video script, and friction logs.
- Git repository pushed and clean on `main` branch.

### ⏳ What is Left Over / Next Polishing Steps for the Next Conversation

- **Demo Video Recording:** Follow the exact 2m45s script in `docs/devpost-submission-package.md` using screen capture.
- **Live Third-Party OAuth:** Replace local SQLite calendar/drive datasets with real Google Cloud Console OAuth 2.0 client IDs if live external sync is desired.
- **Audio Playback Synthesis:** Add Web Speech TTS (Text-to-Speech) so the agent can read aloud final mission results.
- **Dedicated Standalone Repo for Reaper:** Push `packages/mcp-session-reaper` to a separate standalone GitHub repository (optional for the Open Source track).

## 8. Quick Reference / Cheat Sheet for New Conversations

- **MCP Server Port:** `3001` (`http://127.0.0.1:3001/mcp`)
- **Application API Port:** `3000` (`http://127.0.0.1:3000`)
- **Web App Port:** `5173` (`http://127.0.0.1:5173`)
- **Database Path:** `data/alexa_plus.db` (SQLite)
- **User Profile Email:** `shanmukheswaramedicharla@gmail.com`
- **Commands:**
  - Run all tests: `npm test`
  - Build web bundle: `npm run build:web`
  - Run Bedrock test: `npm run test:bedrock`
  - Run Reaper test: `npm run test:reaper`